Atmosic BLE_bridge Example Application Note

Revision History

Date	Version	Description
July 23, 2021	0.50	Initial version created.

CONFIDENTIAL

©2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders. This preliminary document is subject to change without notice.

Table of Contents

1 Overview 3
- 1.1 Hierarchy 5
- 1.2 Hardware Setting 6
- 1.3 Build Example Code 6
- 1.4 Application State 9

2 Helper API Module 10

3 Flash NVDS 10

4 Code Flow 11
- 4.1 Client 11
  - 4.1.1 Initialization 11
  - 4.1.2 Scan 12
  - 4.1.3 Connected 12
  - 4.1.4 Disconnected 12
- 4.2 Server 12
  - 4.2.1 Initialization 13
  - 4.2.2 Advertising 13
  - 4.2.3 Connected 14
  - 4.2.4 Disconnected 14

CONFIDENTIAL

List of Figures

Figure 1 - BLE_bridge Client/Server EVBs and Interface Boards Configuration
Figure 2 - BLE_bridge Example Code Hierarchy
Figure 3 - Interface Board with USB Cables
Figure 4 - Application State Flow Diagram
Figure 5 - Client Code Flow Diagram
Figure 6 - Server Code Flow Diagram

List of Tables

Table 1 - BLE_bridge Module Description
Table 2 - Helper API Module Contents
Table 3 - Flash NVDS Tag

1 Overview

The BLE_bridge example is a flash-based application to show how to send and receive UART0 raw data after Bluetooth LE connection is established. The BLE_bridge example supports server and client roles. This document describes the settings, functionality and flow of BLE_bridge example code.

Default setting of the BLE_bridge - server is as shown below:

UART0:
Flow Control: CTS/RTS is mandatory
Baud Rate: 460800

Legacy advertising Parameter:
Interval minimal: 20 ms (0x00 20)
Interval maximal: 40 ms (0x00 40)
Duration: infinite (0x00)

Advertising data:
Complete Local Name: ATM Bridge
Shortened Local Name: Brdg
(09 FF 24 0A 41 54 4D 42 44 47 0B 09 41 54 4D 20 42 72 69 64 67 65 05 08 42 72 64 67)

The "09 FF 24 0A 41 54 4D 42 44 47" is a specific tag. (Scan)

Scan response data:
Complete Local Name: ATM Bridge
Shortened Local Name: Brdg
(09 FF 24 0A 41 54 4D 42 44 47 0B 09 41 54 4D 20 42 72 69 64 67 65 05 08 42 72 64 67)

Security property:
Service: No Security (0x00 00)
Characteristic: No Security, Write Request and Notification (0x18 00)

The default setting within BLE_bridge - client is as below. Also see Figure 1.

CONFIDENTIAL

UART0 (Connect to HOST):

Flow Control: CTS/RTS is mandatory

Baud Rate: 460800

Scan Parameter:

Scan interval: 10 ms (0x00 10)

Scan window: 10 ms (0x00 10)

Duration: Continuously

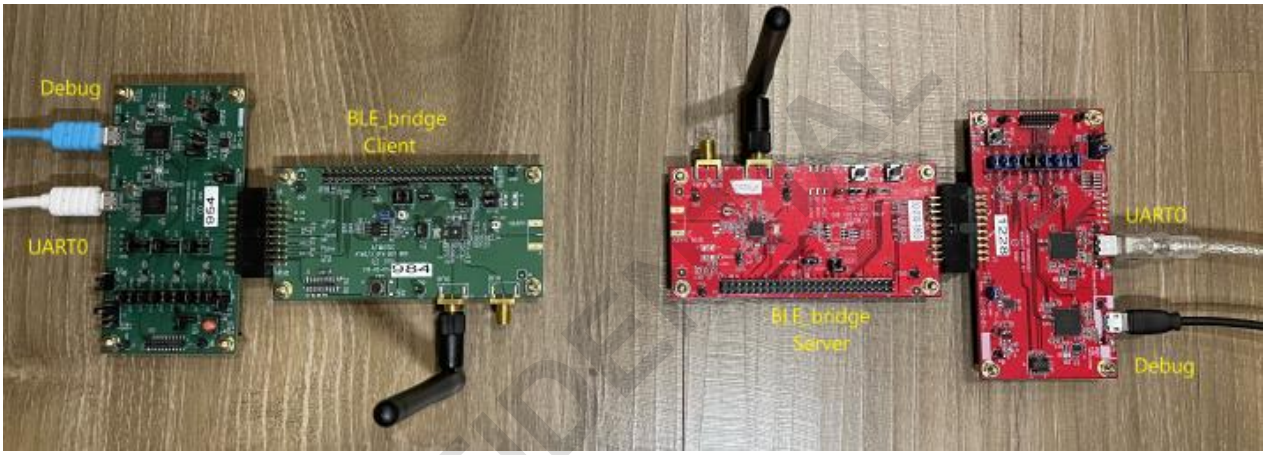


Figure 1 - BLE_bridge Client/Server EVBs and Interface Boards Configuration

CONFIDENTIAL

1.1 Hierarchy

Figure 2 and Table 1 show the BLE_bridge example code hierarchy and BLE_bridge example code description respectively.

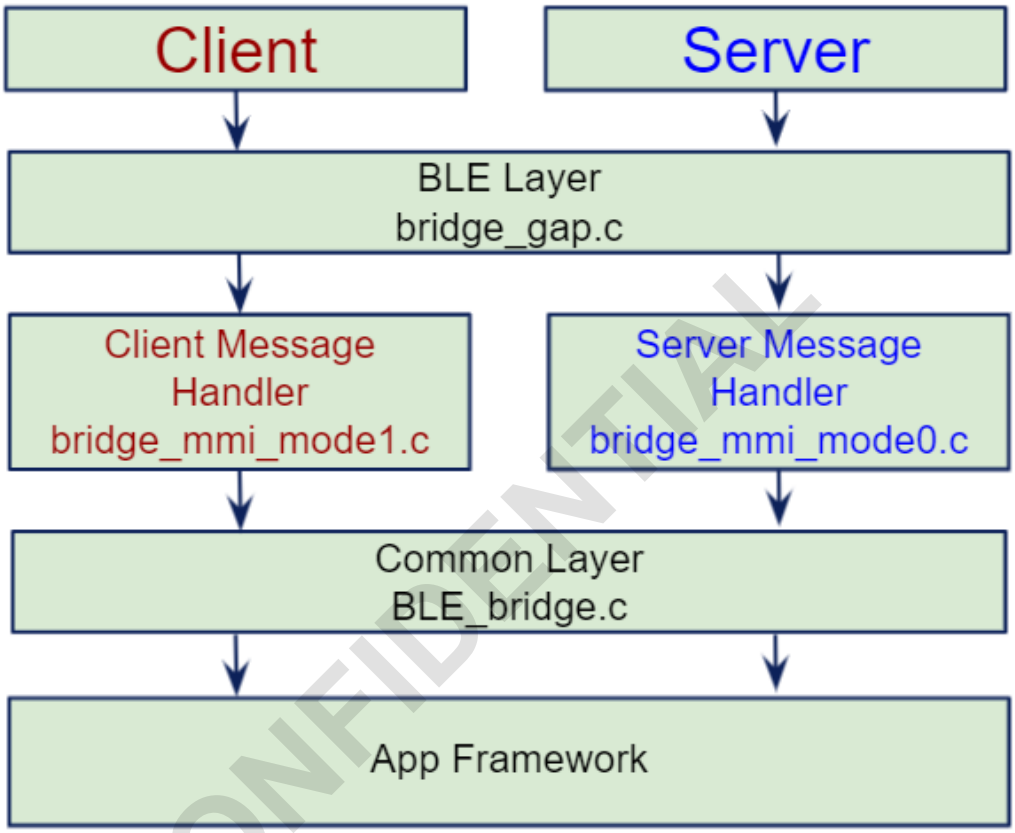


Figure 2 - BLE_bridge Example Code Hierarchy

CONFIDENTIAL

File Name	Description
BLE_bridge.c	The common code of the uart0_raw driver and prf_bridge profile Initialization.
bridge_mmi_mode0.c	Interact with all other modules in example and act as the peripheral of operations.
bridge_mmi_mode1.c	Interact with all other modules in example and act as the center of operations.
bridge_gap.c	Agent of Bluetooth LE layer provides related operations for Bridge client and server while receiving Bluetooth LE events.

Table 1 - BLE_bridge Example Code Description

1.2 Hardware Setting

The interface includes 2 USB ports, USB0 and USB1. USB0 is UART0 port and USB1 is SWD (Serial Wire Debug) port. See Figure 3.



Figure 3 - Interface Board with USB Cables

1.3 Build Example Code

The BLE_bridge example is available in Atmosic SDK 4.1.0. Please go to the example folder named BLE_bridge (platform/atm2/ATM22xx-x0x/examples/BLE_bridge) to generate the firmware.

- Refer to section 1.2 Hardware Setting.
- Go to the BLE_bridge example folder of the Atmosic SDK and apply the following commands
  [Client]

make MODE:=BRIDGE_CLIENT clean
make MODE:=BRIDGE_CLIENT run_all

[Server]
make MODE:=BRIDGE_SERVER clean
make MODE:=BRIDGE_SERVER run_all

- The default Bluetooth BD address is 0x7C4282444700 for the server and 0x7C4282444701 for the client which is predefined in makefile. The MS_BD_ADDR is for the server and the MC_BD_ADDR is for the client. Please apply these compile parameters if need to change the BD address.

  Example: *make MODE:=BRIDGE_SERVER MS_BD_ADDR="00 11 22 33 69 7C" run_all*

- The **Server** console message for different stages
  [Booting]
  1. The uart0_raw interface initializes
  2. The prf_bridge initializes
  3. Automatically enable advertisement (Adv0: ON).

```
@00000037 ATM2202-x0x silicon: 5x5 pkg, Stacked flash
@000000e5 SDK Version: 4.1.0
@0000013d APP Version: 0.0.0.1
Stacked 5x5 EXT_FLASH: 4e 56 44 53 11 06 01 03 12 06 01 01 b5 06 04 00 ...
@00000309 [bridge_gap][N]: bridge_gap_init: CFG_GAP_SEC (0), CFG_GAP_BONDING (0)
@00000474 [ble_gap_se][N]: BOND MASK : 0
@000005bc [  atm_prfs][V]: atm_prfs_init:
@00000660 [   atm_gap][W]: Unhandled GAPM msg 0xd1c
@0000071b [bridge_gap][V]: bridge_gap_init_cfm: status (0)
@00000810 [ble_atmprf][N]: ble_atmprfs_active_svc_db: svc_idx (0), start_hdl (11), attr_num (4)
@00000967 [bridge_gap][V]: bridge_adv_hdlr: act_idx (0), state (2), status (0)
@00000a73 [bridge_gap][V]: bridge_adv_create_cfm: act_idx (0), status (0)
@00000b8c [bridge_gap][V]: bridge_adv_hdlr: act_idx (0), state (4), status (0)
@00000c98 [bridge_gap][V]: bridge_set_adv_data_cfm: act_idx (0), status (0)
@00000db5 [bridge_gap][V]: bridge_adv_hdlr: act_idx (0), state (6), status (0)
@00000ec0 [bridge_gap][V]: bridge_set_scan_data_cfm: act_idx (0), status (0)
@00000fdb [   atm_adv][D]: Adv0: ON (0)
@00001075 [bridge_gap][V]: bridge_adv_hdlr: act_idx (0), state (9), status (0)
```

[BLE Connection]
1. The Bluetooth LE connection created
2. Stop advertisement (Adv0: OFF)
3. MTU exchange

```
@00025dcf [  atm_prfs][V]: atm_prfs_create: conidx (0), task (10), id (118)
@00025ee4 [bridge_gap][V]: bridge_gap_connect_ind: conidx (0)
@00025fbc [   atm_gap][D]: + Peer (0) 7C:42:82:44:47:01
@0002608d [   atm_gap][D]: + Connection interval +    12 (unit:1.25ms)
@00026181 [   atm_gap][D]: + Slave latency       +    10
@0002624a [   atm_gap][D]: + Supervision timeout +   500 (unit:  10ms)
@00026341 [bridge_mmi][N]: link_connected
@000263e6 [   atm_gap][D]: ConnInd idx: 0 role: S
@000264a0 [   atm_adv][D]: Adv0: OFF (0)
@0002653c [bridge_gap][V]: bridge_adv_hdlr: act_idx (0), state (7), status (0)
@00026669 [   atm_gap][W]: Unhandled GAPC msg 0xe2c
@0002673c [bridge_gap][V]: bridge_gap_mtu_exchanged_ind: conidx (0), mtu (247)
@0002684c [bridge_mmi][N]: link_mtu_exchange: mtu (247)
```

- The **Client** console message for different stages
  [Booting]
  1. The uart0_raw interface initial.
  2. The prf_bridge initializes
  3. Start scanning and look for **specific tag** (see 4.1.2 Scan section)

```
@000000ee SDK Version: 4.1.0
@0000014a APP Version: 0.0.0.1
5x5 EXT_FLASH: 4e 56 44 53 11 06 01 03 12 06 01 01 b5 06 04 00 ...
@00000311 [bridge_gap][N]: bridge_gap_init: CFG_GAP_SEC (0), CFG_GAP_BONDING (0)
@0000047f [ble_gap_se][N]: BOND MASK : 0
@000005d9 [  atm_prfs][V]: atm_prfs_init:
@00000683 [   atm_gap][W]: Unhandled GAPM msg 0xd1c
@00000745 [bridge_gap][V]: bridge_gap_init_cfm: status (0)
@00000846 [ble_atmprf][N]: ble_atmprfs_active_svc_db: svc_idx (0), start_hdl (11), attr_num (4)
@000009a4 [bridge_gap][V]: bridge_scan_create_cfm: act_idx (0), status (0)
@00000ab8 [  atm_init][V]: atm_gapm_cmp_evt_create_init(1): status 0
@00000bd1 [bridge_gap][V]: bridge_scan_start_cfm: act_idx (0), status (0)
```

[Bluetooth LE Connection]

1. Found the specific tag during scan
2. Scan stopped
3. The Bluetooth LE connection created
4. Apply the MTU exchange value (default is 247 in makefile)
5. Enable CCCD for transmitting

```
@00143d60 [bridge_gap][V]: bridge_scan_stop_ind: act_idx (0), status (0)
@00143e60 [  atm_scan][D]: Scan0: stopped (0)
@00143f29 [  atm_init][D]: Page(1): ON
@00144279 [  atm_prfs][V]: atm_prfs_create: conidx (0), task (10), id (118)
@0014438d [bridge_gap][V]: bridge_gap_connect_ind: conidx (0)
@00144465 [   atm_gap][D]: + Peer (0) 00:90:4B:03:02:01
@00144537 [   atm_gap][D]: + Connection interval +    12 (unit:1.25ms)
@0014462b [   atm_gap][D]: + Slave latency       +    10
@001446f5 [   atm_gap][D]: + Supervision timeout +   500 (unit:  10ms)
@001447eb [bridge_mmi][N]: bridgec_link_connected:
@001448b4 [   atm_gap][D]: ConnInd idx: 0 role: M
@00144994 [   atm_gap][W]: Unhandled GAPC msg 0xe2c
@001450f1 [bridge_gap][V]: bridge_gap_mtu_exchanged_ind: conidx (0), mtu (247)
@00145201 [bridge_mmi][N]: bridgec_link_mtu_exchange: mtu (247)
@0014db2c [bridge_mmi][N]: bridgec_enable_tx_cb: status (0)
```

- The HOSTs could start to send data between client and server through UART0.

1.4 Application State

Please see Figure 4 for the application state flow diagram.

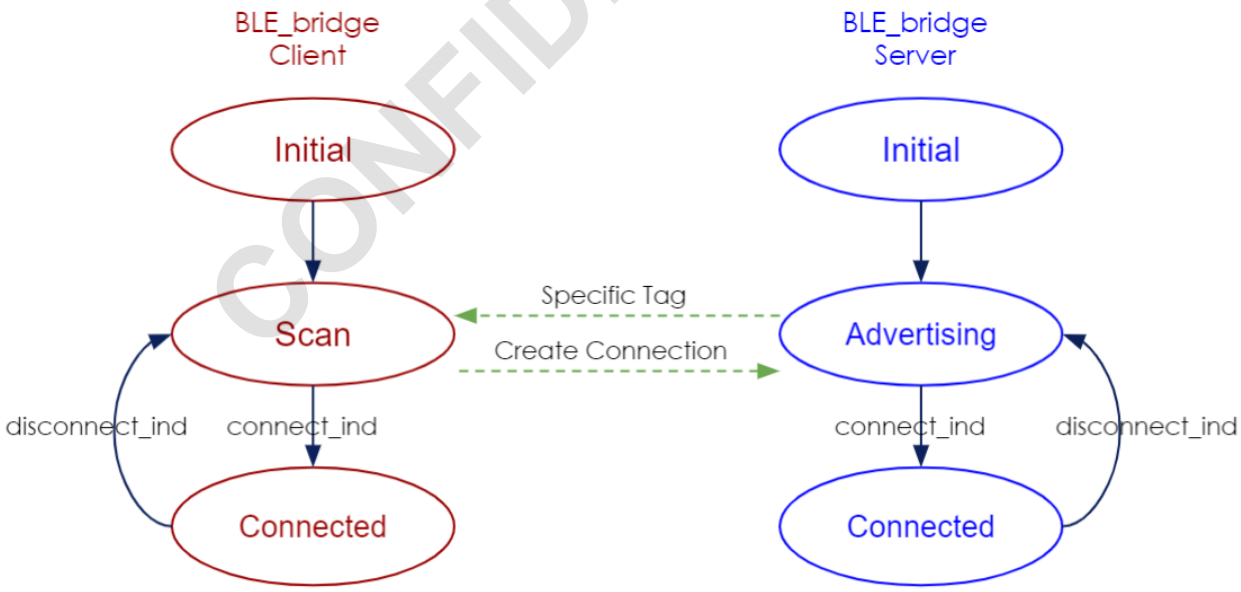


Figure 4 - Application State Flow Diagram

CONFIDENTIAL

2 Helper API Module

Table 2 shows the helper API modules.

File Name	Function Name
BLE_bridge.c	bridge_common_uart0_rx_update() bridge_from_ble() bridge_check_average()
bridge_mmi_mode0.c bridge_mmi_mode1.c	bridge_get_gap_hdlr()
bridge_gap.c	bridge_gap_init() bridge_gap_adv() bridge_gap_get_conidx() bridge_gap_scan() bridge_gap_connect()

Table 2 - Helper API Module Contents

3 Flash NVDS

The following tag data are used in this example. See Table 3.

Tag ID	Description	tds file
0x11	Sleep enable	\tag_data\11-SLEEP_ENABLE\bridge.tds
0x12	Ext. wakeup enable	\tag_data\12-EXT_WAKEUP_ENABLE\bridge.tds

Table 3 - Flash NVDS Tag

How to modify and apply the changes

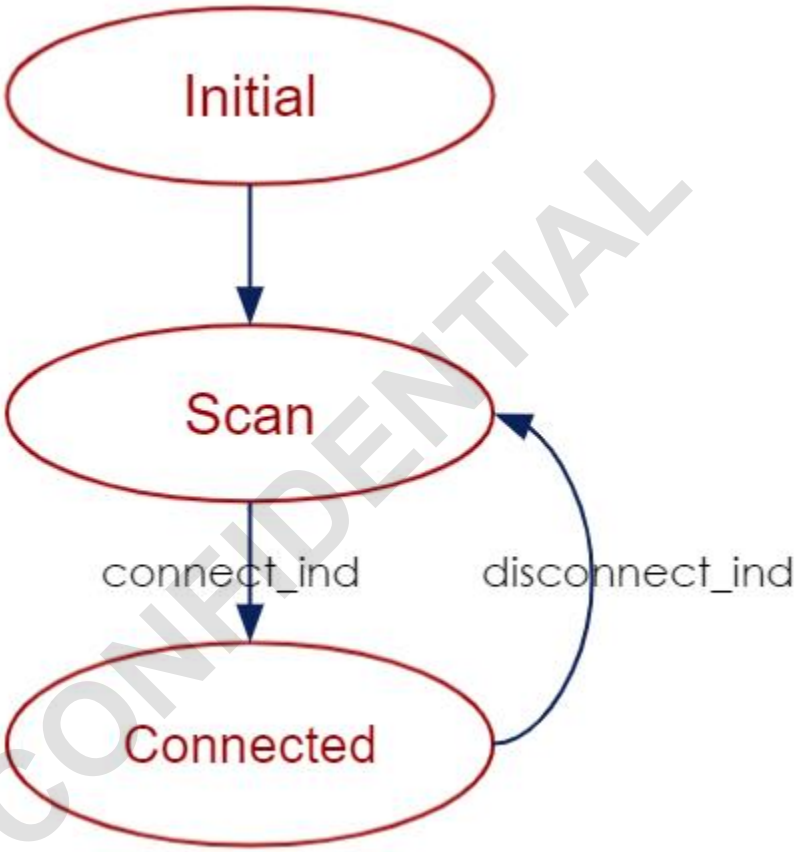
1. Use a text editor tool to open the .tds file to edit.
2. Perform the “make run_all MODE=<BRIDGE_XXXXX>” to apply changes.

CONFIDENTIAL

4 Code Flow

4.1 Client

Please see Figure 5 for the client code flow diagram.



CONFIDENTIAL

Figure 5 - Client Code Flow Diagram

4.1.1 Initialization

1. The *bridge_init()* will be called after system boot-up, which will register the uart0_raw receive callback function, *bridge_from_uart0_cb(),* and disable UART0 receive through *uart0_raw_recv_action_set()* funcion.
2. Invoke *prf_bridge_init()* to initialize the prf_bridge profile.
3. Subsequently, call *bridge_gap_init()* to register the initiate callback function through *atm_init_reg()* and start the Bluetooth LE feature after calling the *atm_gap_init()* function.

4.1.2 Scan

1. The *bridge_gap_init_cfm()* will be called after the Bluetooth LE feature is initialized.
2. Invoke the *atm_scan_create()* to create scan activity and *atm_init_create()* to create initiate activity.
3. The *bridge_scan_create_cfm()* will be called after a scan activity is created. After that, call the *bridge_gap_scan(true)* to start scanning.
4. The `bridgec_adv_report_parsing()` will parse every advertisement to look for a **specific tag** after scanning.

The **specific tag** - **BRIDGE_TAG** is defined in *bridge_mmi_mode1.c* which chooses a part of the sector (include AD length, AD type and AD data) of advertising in BLE_bridge server.

4.1.3 Connected

1. Call the *bridge_gap_scan(false)* after finding the specific tag.The *bridge_gap_connect_ind()* will be called after the connection is established.
2. Invoke *bridge_gap_connect()* to create a connection after getting the *bridge_scan_stop_ind()* event.
3. Subsequently, call the *bridgec_link_connected()* to apply the MTU exchange through *ble_gattc_mtu_exchange()* and enable the Bluetooth LE transmit through *prf_bridgec_enable()*.
4. Next, enable the UART0 receive feature through *uart0_raw_recv_action_set()*.
5. Now, send data to service via UART0 from HOST.

4.1.4 Disconnected

1. The *bridge_gap_disconnect_ind()* will be called after the Bluetooth LE disconnect occurs.
2. Next, call *bridgec_link_disconnected()* to turn on the scan through *bridge_gap_scan(true)* and disable the UART0 receive feature through *uart0_raw_recv_action_set()*.

4.2 Server

Please see Figure 6 for the server code flow diagram.

CONFIDENTIAL

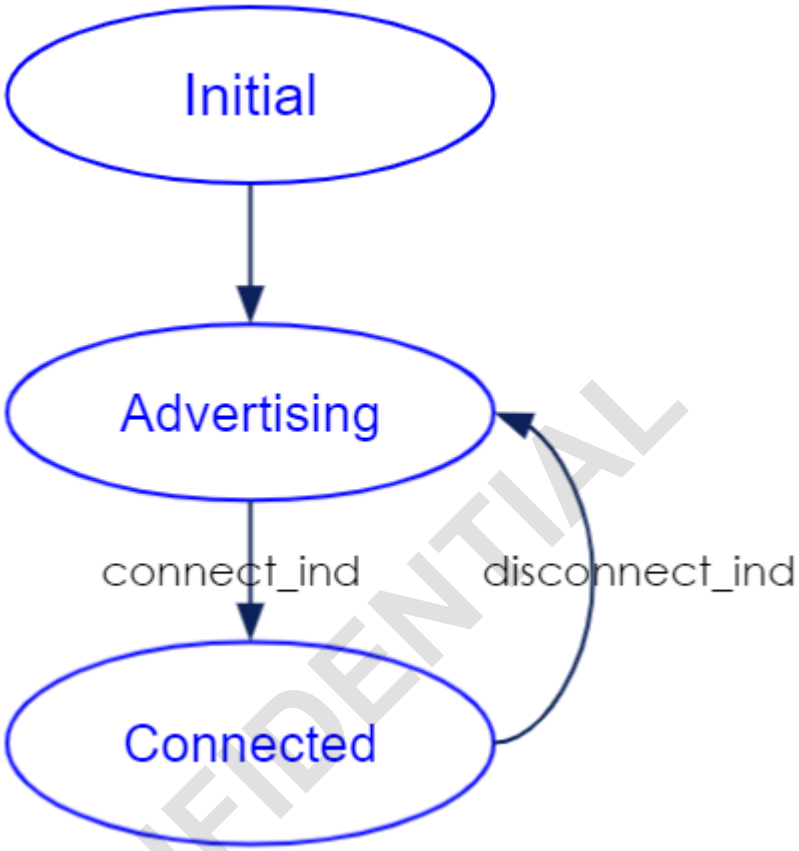


Figure 6 - Server Code Flow Diagram

CONFIDENTIAL

4.2.1 Initialization

1. The *bridge_init()* will be called after system boot-up, which will register the uart0_raw receive callback function, *bridge_from_uart0_cb(),* and disable UART0 receive through *uart0_raw_recv_action_set()* funcion.
2. Invoke *prf_bridge_init()* to initialize the prf_bridge profile.
3. Subsequently, call *bridge_gap_init()* to register the initiate callback function through *atm_adv_reg()* and start the Bluetooth LE feature after calling the *atm_gap_init()* function.

4.2.2 Advertising

1. The *bridge_gap_init_cfm()* will be called after the Bluetooth LE feature is initialized.

2. Invoke the *atm_adv_create()* to create advertising activity.
3. The *bridge_adv_create_cfm()* will be called when the advertising activity is completed.
4. Subsequently, call *atm_adv_set_adv_data()* to set the advertising data and call *atm_adv_set_scan_data()* to set the scan response data.

   NOTE: *You can modify the advertising data or scan response data in this stage.*

5. The `bridge_set_adv_data_cfm()` will be called when setting advertising data is completed.
6. Next, `call` *bridge_gap_adv(true)* to start advertising.

4.2.3 Connected

1. The *bridge_gap_connect_ind()* will be called after the connection is established.
2. Subsequently, call the *link_connected()* after receiving the *bridge_connect_ind()*.
3. Next, enable the UART0 receive feature through *uart0_raw_recv_action_set()*.
4. Now, you can send data to service via UART0 from HOST.

4.2.4 Disconnected

1. The *bridge_gap_disconnect_ind()* will be called after Bluetooth LE disconnect occurs.
2. Next, call *link_disconnected()* to start advertising through *bridge_gap_adv(true)* and disable the UART0 receive feature through *uart0_raw_recv_action_set()*.

CONFIDENTIAL



ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2021 by Atmosic Technologies. All rights reserved.

CONFIDENTIAL

www.atmosic.com